

Smart MCQ Solver Challenge

DL and GenAI Project Report

BSDA2001P: Introduction to DL and GenAI

Roll Number: 23f2004192

1. Abstract

This report covers our work on the Kaggle Smart MCQ Solver Challenge. The task was to build a system that can pick the right answer for multiple choice questions. Each question has five options (A through E), and the system needs to rank the top 3 answers. We tried three different models. First, we built a Bi-LSTM model from scratch. Second, we fine-tuned a pre-trained DistilBERT model. Third, we used a Sentence Transformer (all-MiniLM-L6-v2) with a Logistic Regression classifier as our model of choice. The DistilBERT model gave the best results with a validation MAP@3 of 0.6068. The Sentence Transformer + LR model scored 0.4338, and the scratch BiLSTM scored 0.5358. All experiments were tracked using Weights and Biases (wandb). This report goes through the dataset, preprocessing, tokenization, model details, and a comparison of all results.

2. Introduction

Problem Statement

The Kaggle Smart MCQ Solver Challenge asks us to answer science-based multiple choice questions. Each question has a prompt and five answer choices labeled A, B, C, D, and E. We need to predict the correct answer. The competition metric is MAP@3, which means we submit our top 3 guesses and get a higher score if the correct answer appears earlier in the list.

Project Objective

Our goals for this project were: (1) learn how to work with text data for deep learning, (2) build a model from scratch to understand the basics, (3) fine-tune a pre-trained transformer to see how transfer learning helps, (4) try a different approach using sentence embeddings, and (5) compare all three approaches fairly.

Report Structure

The rest of this report is organized as follows. Section 3 describes the dataset and how we cleaned the data. Section 4 talks about the tokenization strategies. Section 5 explains the three models in detail. Section 6 compares their performance. Section 7 wraps up with conclusions and ideas for future work.

3. Dataset and Preprocessing

Dataset Description

The dataset comes from the Kaggle competition. The training set has 2000 rows and the test set has 500 rows. Each row has a prompt (the question text) and five answer options (A, B, C, D, E). The training set also has the correct answer column. The questions cover a wide range of science topics.

Exploratory Data Analysis

We looked at the distribution of correct answers across A to E and found it was roughly balanced. We also checked the text lengths of prompts and options. Most prompts are under 100 words, though some can be much longer. The options are usually shorter, around 10 to 40 words each.

Data Preprocessing

For cleaning, we converted all text to lowercase and removed special characters using regular expressions. Extra whitespace was also stripped. For the prompt field, we removed boilerplate phrases like "Pick the best possible answer" since they do not help the model learn anything useful.

Data Splitting (Leak-Free)

A big concern was data leakage. Some questions in the dataset share the same set of answer options but with slightly different wording. If these end up in both training and validation sets, the model could cheat by memorizing the option patterns. To fix this, we used a UnionFind algorithm to group questions that share prompts or option sets. Then we used GroupShuffleSplit with a 85/15 ratio so that no group appears in both splits. This gave us 1688 training samples and 312 validation samples with zero overlap.

4. Tokenization Strategy

BiLSTM (Custom Word Tokenizer)

For the scratch BiLSTM model, we built a simple whitespace-based tokenizer. After cleaning the text, we split it on spaces. We counted word frequencies from the training set and kept the top 15,000 words (though only 2,701 unique words appeared). We used index 0 for padding and index 1 for unknown words. Prompts were truncated to 128 tokens and options to 64 tokens.

DistilBERT (WordPiece Tokenizer)

For the DistilBERT model, we used the WordPiece tokenizer from Hugging Face. This tokenizer breaks words into sub-word pieces, which helps the model handle words it has never seen before. For example, "acceleration" might become ["accel", "##eration"]. We set the maximum sequence length based on the 95th percentile of the training data lengths. Sequences shorter than this were padded, and longer ones were truncated.

Sentence Transformer

The Sentence Transformer (all-MiniLM-L6-v2) has its own built-in tokenizer. It handles tokenization internally when we call the encode function. We did not need to set any special tokenization parameters for this model.

Comparison

The WordPiece tokenizer used by DistilBERT was the most effective choice for our task. It can handle out-of-vocabulary words by breaking them into known sub-word pieces. Our custom word tokenizer for the BiLSTM was simple but loses information on unknown words. We chose WordPiece for the best model because it was designed to work with the pre-trained weights.

5. Modeling and Experimentation

5.1 Model 1: Fine-Tuned Transformer (DistilBERT)

We chose DistilBERT (distilbert-base-uncased) as our fine-tuned transformer model. DistilBERT is a distilled version of BERT that keeps about 97% of BERT's language understanding while being 60% faster and 40% smaller (Sanh et al., 2019). It has 6 transformer layers, a hidden size of 768, and 12 attention heads.

Architecture: We used `AutoModelForMultipleChoice` from Hugging Face. This wraps the DistilBERT encoder with a classification head that scores each of the 5 options. The model takes in the prompt paired with each option and outputs logits for all five choices.

Fine-Tuning Details:

- Optimizer: AdamW
- Learning rate: 1.5e-5
- Scheduler: Linear warmup (10% of total steps)
- Epochs: 6
- Batch size: 4
- Gradient clipping: max norm 1.0
- Cross-validation: 5-fold GroupKFold
- Best model saved by MAP@3 on fold validation set

Retrieval-Augmented Inference: On top of DistilBERT, we also built a retrieval system. We used the all-MiniLM-L6-v2 sentence transformer to embed all training prompts into a FAISS index. At inference time, for each test question, we retrieve the top-15 most similar training questions and build a probability distribution from their answer labels. We then blend the DistilBERT probabilities with the retrieval probabilities. The best blend weight was $\alpha=0.95$, meaning the model mostly trusts DistilBERT but gets a small boost from retrieval.

5.2 Model 2: Model of Choice (Sentence Transformer + Logistic Regression)

For our model of choice, we used the all-MiniLM-L6-v2 sentence transformer (Reimers and Gurevych, 2019). Sentence transformers produce fixed-size vector representations of sentences. The idea here was to turn the MCQ problem into a feature-based classification task.

Feature Extraction: For each question, we encoded the prompt and each of the 5 options separately using the sentence transformer. For each prompt-option pair, we concatenated the prompt embedding, the option embedding, and the cosine similarity between them. This gave us a feature vector for each question. We then applied PCA to reduce the feature dimensions to 256 components.

Classifier: We trained a Logistic Regression classifier (multinomial, solver=lbfgs, C=1.0, max_iter=1000) on these features. We used 5-fold GroupKFold cross-validation, same as the other models.

We picked this as our model of choice because it is very fast to train. The sentence transformer produces the features in one pass, and the Logistic Regression fits in seconds. This makes it easy to experiment quickly.

5.3 Model 3: RNN-Based Model (Scratch BiLSTM)

For our scratch model, we built a Bidirectional LSTM from the ground up using PyTorch. A BiLSTM reads the input sequence in both directions (forward and backward), which helps it capture context from both sides of each word (Hochreiter and Schmidhuber, 1997).

Architecture:

- Embedding layer: vocab size 2701, embedding dim 128 (trained from scratch)
- BiLSTM: 2 layers, hidden size 128, bidirectional (output dim = 256)
- Attention Pooling: learns which time steps matter most
- Classifier: Linear(768, 128) -> ReLU -> Dropout(0.3) -> Linear(128, 1)
- The classifier takes [prompt_repr, option_repr, prompt*option] as input

Training Details:

- Optimizer: Adam (lr=1e-3, weight_decay=1e-4)
- Scheduler: CosineAnnealingLR
- Epochs: 15
- Batch size: 32
- Gradient clipping: max norm 1.0
- Cross-validation: 5-fold GroupKFold
- Loss: CrossEntropyLoss

The model encodes the prompt and each option separately through the same BiLSTM encoder. For each option, it concatenates the prompt representation, the option representation, and their element-wise product. A classifier head then scores each option. The option with the highest score is the predicted answer.

6. Performance and Comparative Analysis

Evaluation Metrics

We used three metrics to evaluate our models:

- MAP@3 (Mean Average Precision at 3): the main competition metric. We submit our top 3 guesses and get scored based on where the correct answer appears.
- Accuracy: the fraction of questions where the top-1 prediction is correct.
- Macro F1: the average F1 score across all 5 classes.

Training Performance: BiLSTM (Scratch)

The BiLSTM model trained for 15 epochs across 5 folds. The training loss dropped quickly from around 1.47 to below 0.06, but the validation loss kept going up, which shows clear overfitting. The best fold MAP@3 values were: Fold 0: 0.5710, Fold 1: 0.5256, Fold 2: 0.7046, Fold 3: 0.6864, Fold 4: 0.6434.

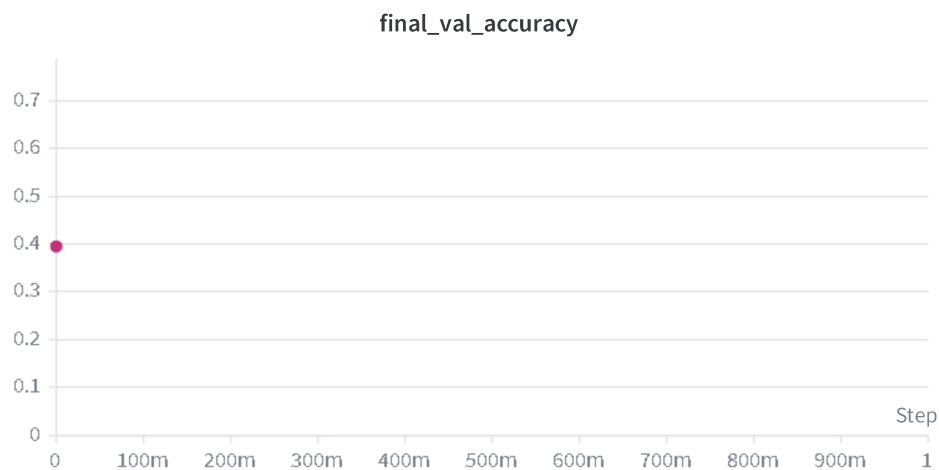


Figure 1: BiLSTM Validation Accuracy (wandb)

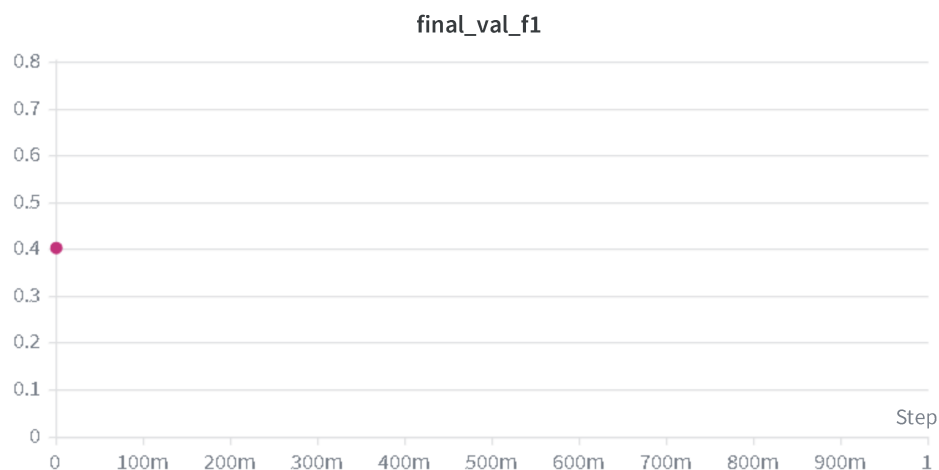


Figure 2: BiLSTM Validation F1 Score (wandb)

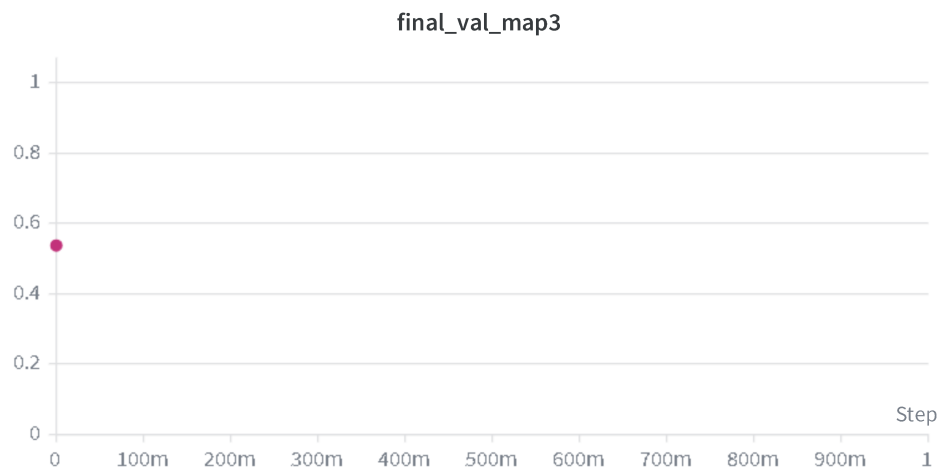


Figure 3: BiLSTM Validation MAP@3 (wandb)

Training Performance: DistilBERT

The DistilBERT model trained for 6 epochs across 5 folds. The training loss dropped steadily from around 1.4 to below 0.05. Validation metrics improved across epochs. The best fold MAP@3 values were: Fold 0: 0.5503, Fold 1: 0.5247, Fold 2: 0.6691, Fold 3: 0.5964, Fold 4: 0.7280.

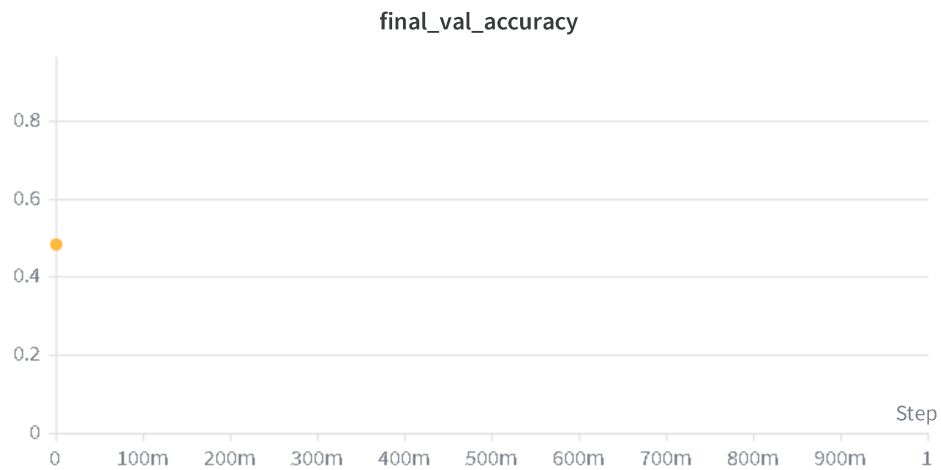


Figure 4: DistilBERT Validation Accuracy (wandb)

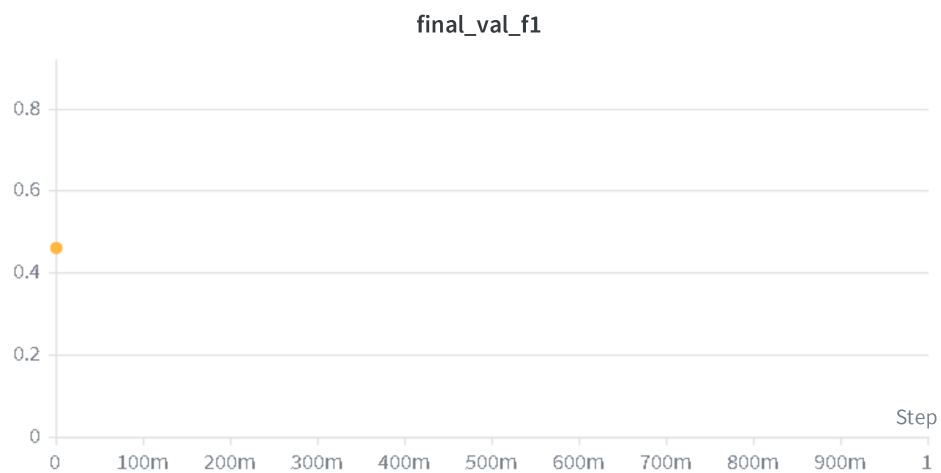


Figure 5: DistilBERT Validation F1 Score (wandb)

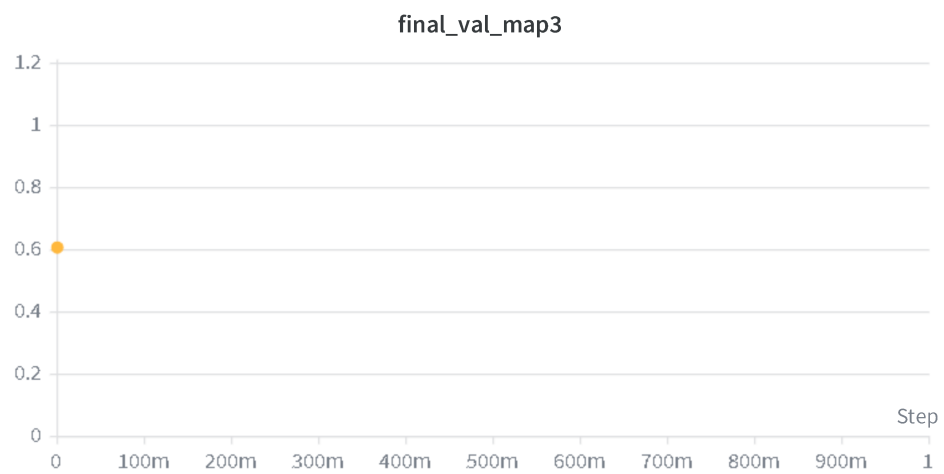


Figure 6: DistilBERT Validation MAP@3 (wandb)

Training Performance: Sentence Transformer + LR

The Sentence Transformer model used frozen embeddings from all-MiniLM-L6-v2 with a Logistic Regression classifier on top. The fold-level MAP@3 values were: Fold 0: 0.5385, Fold 1: 0.5523, Fold 2: 0.4975, Fold 3: 0.5292, Fold 4: 0.5302.

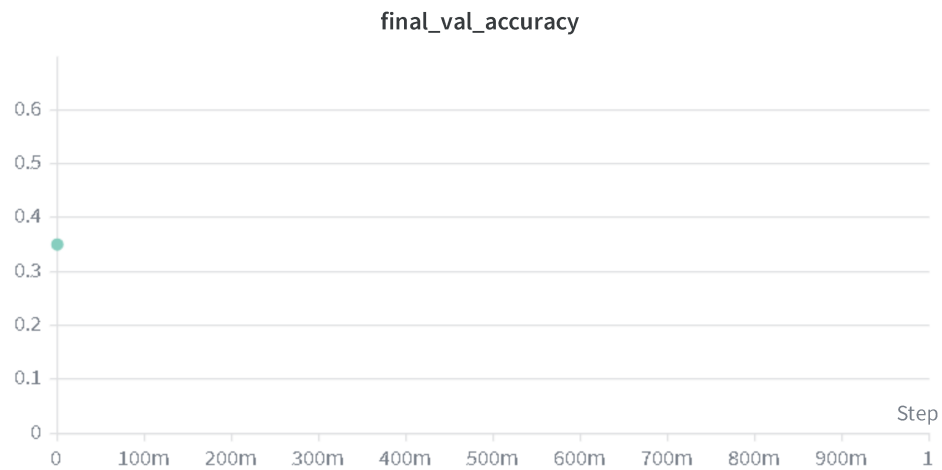


Figure 7: Sentence Transformer Validation Accuracy (wandb)

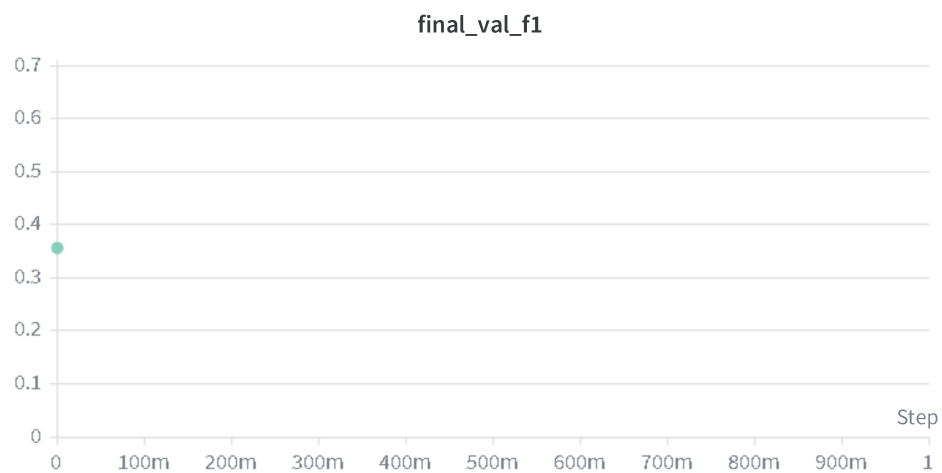


Figure 8: Sentence Transformer Validation F1 Score (wandb)

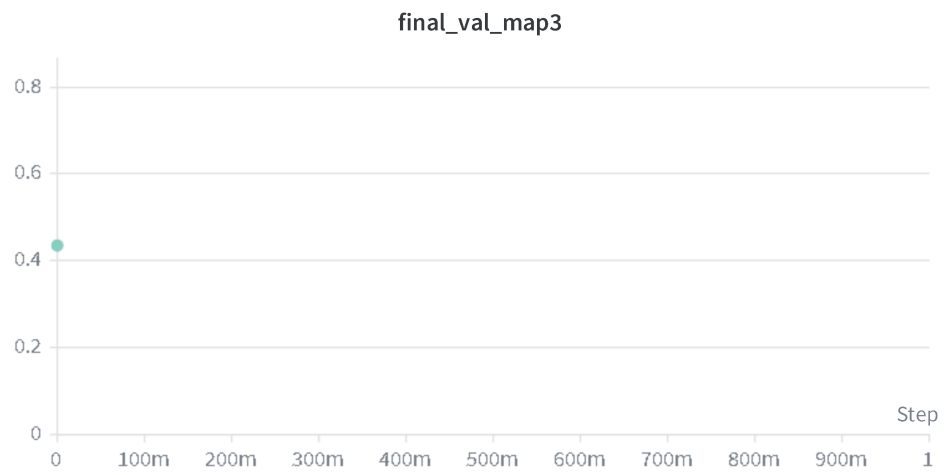


Figure 9: Sentence Transformer Validation MAP@3 (wandb)

Comparative Summary

The table below compares the final holdout validation results for all three models. All numbers are from the same 312-sample holdout validation set.

Model	MAP@3	Accuracy	Macro F1
BiLSTM (Scratch)	0.5358	0.3942	0.4032
DistilBERT + Retrieval	0.6068	0.4840	0.4604
Sentence Trans + LR	0.4338	0.3494	0.3556

DistilBERT performed the best on all three metrics. This makes sense because it was pre-trained on a huge text corpus and has a deep understanding of language. The retrieval augmentation gave it a small extra boost (from 0.6031 to 0.6068). The BiLSTM came in second on MAP@3, which is reasonable since it can still capture sequence patterns, but it lacks the pre-trained knowledge. The Sentence Transformer + LR model scored the lowest. Even though the sentence embeddings are good, the Logistic Regression classifier is quite simple and cannot capture the complex relationships between prompts and options as well as the other models.

In terms of training time, the Sentence Transformer + LR was the fastest (under a minute). The BiLSTM took about 6 minutes total. The DistilBERT took the longest at around 25 minutes. So there is a clear tradeoff between performance and compute cost.

7. Conclusion and Future Work

Key Learnings

This project taught us a lot about applying deep learning to NLP problems. We learned how to build a model from scratch using PyTorch, including writing custom datasets, collate functions, and training loops. We also learned how to fine-tune large pre-trained models using Hugging Face, and how to use sentence embeddings for feature extraction.

One important lesson was the value of pre-training. The DistilBERT model, which was pre-trained on a large corpus, significantly outperformed the BiLSTM, which had to learn everything from just 2000 samples. This clearly shows why transfer learning is so popular in NLP.

Challenges Faced

The biggest challenge was overfitting. With only 2000 training samples, all models tended to memorize the training data. We tried to handle this with dropout, gradient clipping, and early stopping based on MAP@3. Another challenge was data leakage. Some questions shared the same option sets, and we had to use the UnionFind grouping trick to make a fair train/val split.

Future Work

There are several things we could try to improve the results:

- Use a larger model like DeBERTa-v3 or RoBERTa-large, which might capture more nuance.
- Build an ensemble that combines the predictions of all three models.
- Try data augmentation, like paraphrasing questions or shuffling option order.
- Experiment with different learning rate schedules and hyperparameter tuning.

8. References

- [1] Sanh, V., Debut, L., Chaumond, J., and Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.
- [2] Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).
- [3] Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780.
- [4] Devlin, J., Chang, M.W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT 2019.